Bar-omètre — Technical Architecture Document

EPITA Android Project · The Spice Girls Two Nice Guys · 25 May 2026

1. Architecture Overview

Bar-omètre is a Java Android application that helps users discover bars in Paris. The app uses multiple Activities and Fragments with a layered architecture composed of presentation, repository, and SQLite data layers.

2. Main Components

Component	Type	Requirement
MainActivity	Activity	R1, R2
BarListFragment	Fragment	R4, R5
BarDetailActivity	Activity	R1, R2, R3
FavoritesActivity	Activity	R8
FilterFragment	Fragment	R3, R7

3. Data Model

The common model used throughout the project is called Bar.java. Bar details like name, address, GPS coordinates, rating, tags, website, hours of operation, and favourite status are all stored in it. Serialisable for Intent and Fragment data transmission is implemented by the model.

4. Requirement Coverage

Requirement	Implementation
R1	Implicit Intents for website, maps, sharing, and phone dial.
R2	Landscape layouts and state preservation with onSaveInstanceState().
R3	Fragments with Bundle-based data exchange.
R4	Toolbar and context menu actions.
R5	RecyclerView with custom bar cards.
R6	Dark/light themes with themes.xml.
R7	GPS using FusedLocationProviderClient.
R8	BroadcastReceiver for offline mode.

5. Person 1(Karma) — Bar Discovery & Map navigation and shared UI architecture

The Bar Discovery module is responsible for the application's main navigation flow and interactive map experience. MainActivity serves as the primary entry point of the application and hosts the fragment-based interface used throughout the project. Google Maps SDK was integrated to provide an interactive visualization of bars across Paris, where each establishment is dynamically displayed as a map marker loaded through the shared repository layer. Users can interact directly with markers to select bars and navigate toward the detailed information screen through explicit Intent communication with BarDetailActivity.

The module also manages responsive orientation handling by providing separate landscape layouts that display map and detail content simultaneously, improving usability on horizontal screens and satisfying Android configuration adaptation requirements. A shared ViewModel architecture based on LiveData observers was implemented to

synchronize map updates, marker selections, filtering operations, and future GPS integration between fragments and activities while remaining lifecycle-aware. In addition, the module integrates toolbar menus, offline state indicators.

The overall architecture was designed to decouple UI logic from data handling through a repository pattern and shared ViewModel communication layer.

6. Person 3(Peushi) — Bar Detail & Sharing

The Bar Detail module is in charge of presenting all actions a user can do on a selected bar as well as full information about it. As the host container, BarDetailActivity configures the toolbar, delegates UI rendering to BarDetailFragment via a FragmentTransaction, and receives the chosen Bar object from MainActivity or BarListFragment via a Serialisable Intent extra.

The bar name, rating via a RatingBar, entire address assembled by Bar.getFullAddress(), phone number, and opening hours are all bound to the layout at runtime by BarDetailFragment using ViewBinding (FragmentBarDetailBinding). Similar to the original conditional rendering logic of the Node app, views containing possibly null data (phone, hours) are conditionally displayed and set to View.GONE by default. In keeping with the original web frontend, bar images are loaded asynchronously using Glide and the loremflickr service.

Four implicit intents are implemented by the module: ACTION_VIEW with a Google and an HTTPS URI for webpage navigation.navigation: ACTION_DIAL with a tel: URI for the phone dialler, ACTION_SEND with text/plain for bar sharing via a system chooser, and URI scheme for Google Maps turn-by-turn instructions. Orientation handling is controlled by the savedInstanceState == null guard to avoid fragment double-stacking during rotation and onSaveInstanceState() for state preservation.

The contextual menu need is implemented by a long-press AlertDialog on the bar card, which provides Add to Favourites and Share options. The feature is permanent between sessions because the favourites action is completely linked to BarRepository, calling addFavorite() or removeFavorite() on the SQLite database based on the current isFavorite() state. All interactions inside the Fragment lifecycle share the BarRepository instance, which is initialised in onCreate().

7. Person 4 Jakub - Favorites & Offline Storage

For the database, we used BarDbHelper (which extends SQLiteOpenHelper) to manage a bars table. It tracks all the bar details plus is_favorite and last_viewed stats. On the very first launch, onCreate seeds the database by reading paris_bars.json from the raw resources using a BufferedReader so it doesn't crash on large files, inserting all 67 bars in one big transaction. We also added a type column in version 2 to make filtering easier, and wrote some static helper methods so we can safely read data from Cursors across the app.

Everything goes through BarRepository, which implements IBarRepository to be our single source of truth. It handles all the queries like searching by name, sorting by ratings, and filtering by city. To track history, whenever getBarById is called, it saves the current time into COLUMN_LAST_VIEWED so getRecentlyViewedBars can show the user's recent history. Favoriting just toggles the is_favorite column, and cacheBar uses CONFLICT_REPLACE so offline saving doesn't create duplicate rows.

On the UI side, FavoritesActivity and FavoritesFragment pull data from getFavoriteBars and automatically refresh inside onResume, while BarDetailFragment hooks the favorite button directly to the repository.

To handle internet drops, we made a ConnectivityReceiver that gets registered dynamically in MainActivity's lifecycle. When the connection changes, it tells BarViewModel to switch between live and cached data and toggles the offline banner. Finally, BarViewModel.init lazily swaps out our temporary FakeRepository for the real SQLite one on startup, so the whole app gets real data without needing any code changes

8. Permissions & Libraries

The app uses INTERNET, ACCESS_NETWORK_STATE, and GPS permissions for maps, image loading, and nearby bar features. External libraries include Glide for image loading, Google Maps SDK, RecyclerView, Lifecycle components, and Material Components.

9. Team Contributions

Person	Name	Contribution
P1	Karma	Maps and MainActivity
P2	Tristan	RecyclerView and Search
P3	Peushi	Bar Detail & Intents
P4	Jakub	SQLite & Favorites
P5	Sasha	GPS, Filters & Themes